

Graphle

Graph-oriented file management system

Organize your files the way you think.

Filip Bubák

Supervisor: Ing. Pavel Koupil, Ph.D.

Motivation

Operating systems store files in a strict tree of directories, where every file lives in exactly one place. Yet a single photograph may belong to a holiday, a person, and a project at once. Associations like this a tree simply cannot express. Existing tools bridge this gap only partially, and rarely let users freely connect arbitrary files with typed relationships and tags across the whole filesystem in an easily queryable way.

Goals

- Graph data model** — represent files and folders as nodes in a labeled property graph, joined by user-defined typed relationships and tags, while staying backward-compatible with the existing filesystem.
- Query language (DSL)** — search files and manipulate their tags and relationships by graph structure rather than by path alone.
- Backend** — maintain the graph and keep it consistent with a live filesystem modified from outside Graphle.
- Remote access** — connect the graphical client to a remote backend and browse and manage its files just as if Graphle was running locally.
- Graphical client** — browse, open, move and delete files, edit tags and relationships, and write and run DSL queries with filename autocomplete in one place.

Query language

A compact DSL exposes every graph operation, both inside the GUI and as a standalone tool for scripts. Searches are built from scopes: file scopes use (...), relationship scopes use [...].

Find every file tagged as a dataset:

```
find (tagName = "type" AND tagValue = "dataset")
```

Follow a typed relationship to its target files:

```
find (location = ".../research-questions.md")[name = "motivates"]()
```

Add a key-value tag to a file:

```
addTag ".../the-matrix.mkv" "year" "1999"
```

Add a typed relationship between two files:

```
addRel ".../research-questions.md" ".../experiment-plan.md" "motivates"
```

- Operators = != <> < <= > >=, combined with AND / OR.
- [DESC] and [PRED] expose the live filesystem hierarchy — usable even for files not yet registered in Graphle.

Conclusion

Graphle shows that files can stay in the existing filesystem while being organised and explored as a graph. The result is a working application built around a graph data model, a query DSL, a backend exposing GraphQL, REST and WebSocket interfaces, and a cross-platform graphical client.

Acknowledgements

I would like to thank my supervisor, Pavel Koupil, for his guidance and support throughout this thesis.

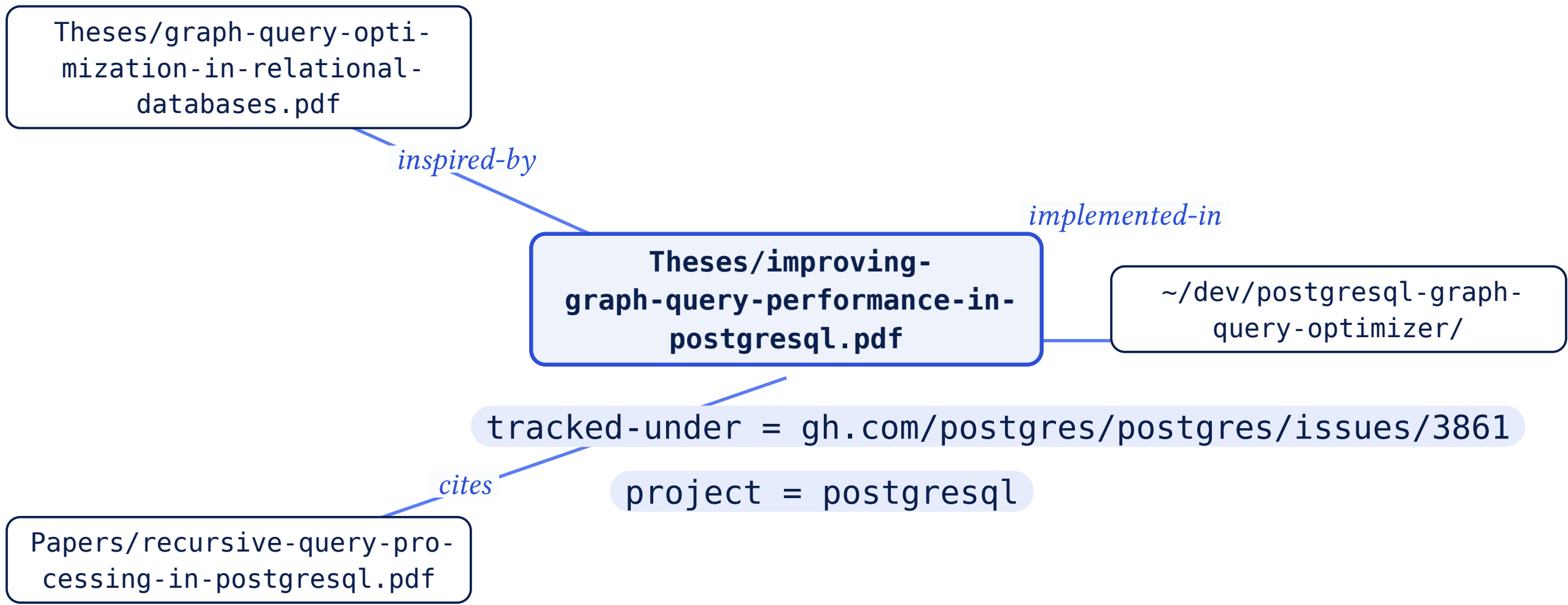
Additional information



GitHub github.com/TrueBubo/Graphle
Email bubak.filip@pm.me
LinkedIn [linkedin.com/in/filip-bubak](https://www.linkedin.com/in/filip-bubak)

How it works

Graphle adds a **semantic graph layer** on top of an existing filesystem. Files stay in their original directories and remain usable by ordinary tools. Only the user's semantic metadata, **tags** and typed **relationships**, are persisted in a labeled property graph. Parent and descendant edges are derived live from the filesystem, so no data already maintained by the OS is duplicated.

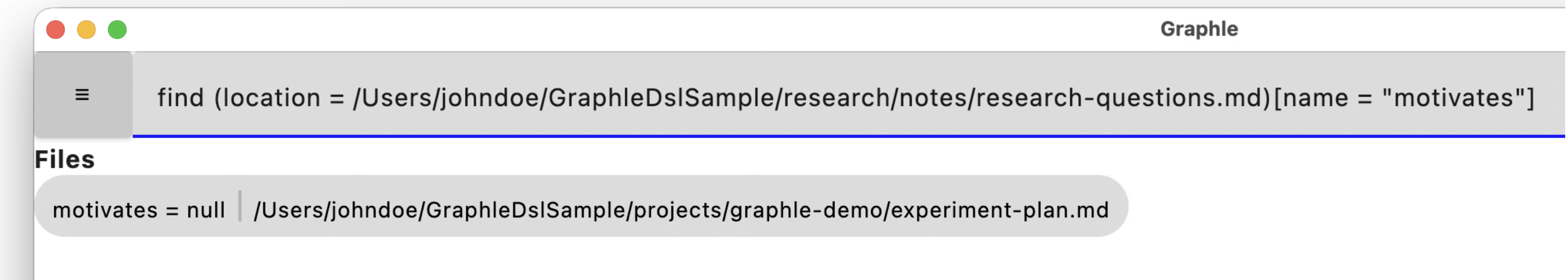
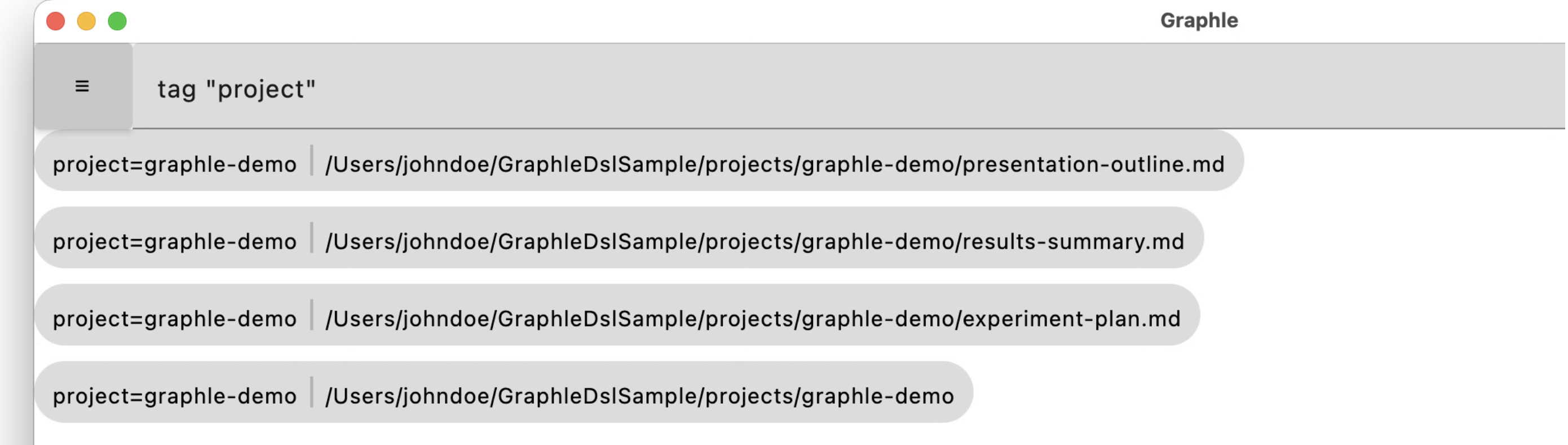
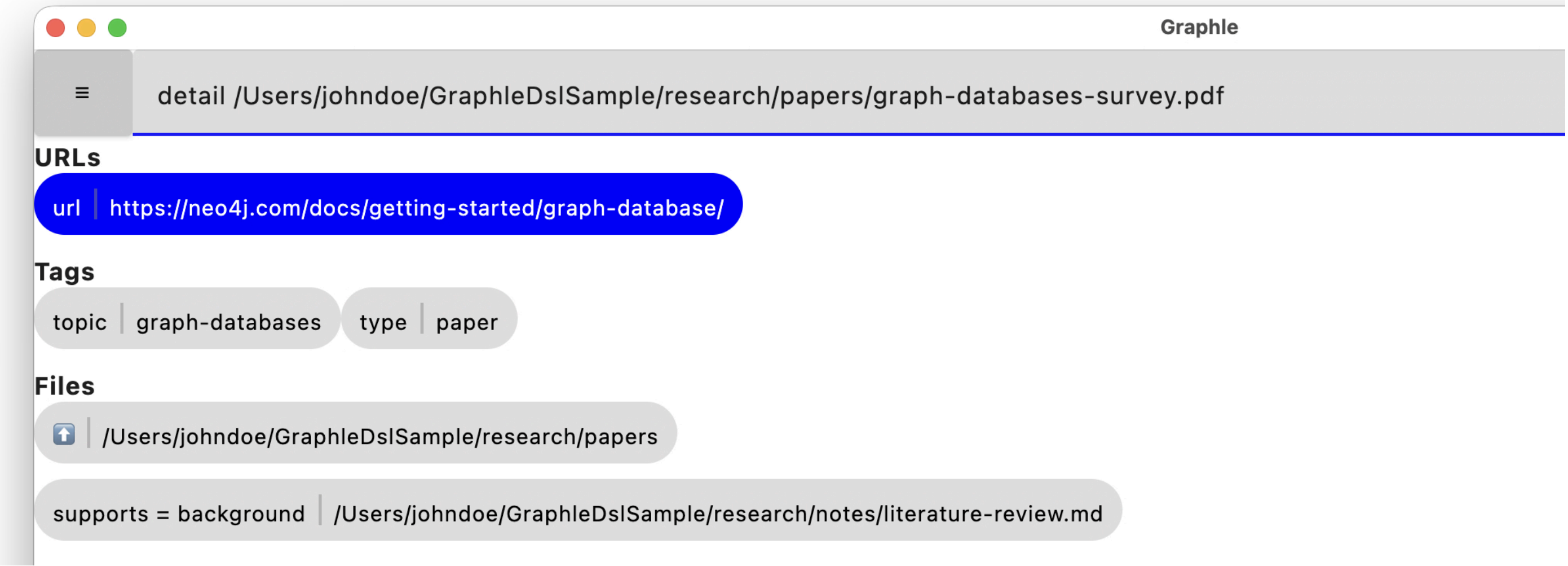


Unlike a tree, one paper can support multiple theses while the work links directly to its implementation.

Files are loaded into the graph lazily on first visit, so startup needs no full disk scan, and a background sweeper prunes entries for files that have disappeared, keeping the graph consistent after external changes.

The application

A cross-platform desktop client organises every operation around the currently selected file. The hamburger menu provides Home, Trash, hidden-file and dark-mode controls, together with actions for the current file. A synchronised command line always shows the DSL query behind the view.



Top to bottom: a file with its tags and relationships · files carrying a given tag · results of a relationship traversal.